

Software-based Modelling Tools

Is it not faster to draw a picture using software tools like Paint than specify it as 0's and 1's? That is how simple, swift and flawless the design and development of embedded systems has become now, all thanks to software-based modelling tools that pervade the entire lifecycle from requirements gathering to implementation



■ **JANANI GOPALAKRISHNAN
VIKRAM, WITH INPUTS FROM
ONKAR SHARMA**

The chances of introducing errors early in the lifecycle and discovering them late are quite high in the traditional models for embedded systems design and development. What follows next is the same old story. Go back, fix the error, redo everything from the erroneous point onwards, and get pulled up by the boss or client for cost and time overruns! “Did I forget to offer prayers at the requirements gathering stage, or did I choose the wrong approach?”—you

wonder. It could be either.

Next time, however, make sure that the latter possibility does not arise, by turning to a model-based approach to embedded systems design and development. This is a very attractive approach, especially when time-to-market is very critical—a pre-condition that applies to most systems today!

What's model-based development?

As the name suggests, in model-based development, explicit graphical models are used to define and describe a system. To be more accurate, you do not even describe the system, instead you

mathematically model the system's behaviour, i.e., you specify how it should perform. The system is then designed using simulation, tested and validated in real time, and implemented.

While there are individual tools for each stage of the lifecycle too, the trend is in favour of tools that span the entire process of systems design and development. So how does a holistic modelling tool work? Basically, requirements are gathered and expressed as a model using block diagrams. The model is refined by adding details relevant to the specific domains they might be implemented in. It is interesting to note here that the entire system, including environmental models, mechanical and electrical components, supervisory logic and control algorithms, is designed in an integral fashion. Intellectual property and engineering data is reused from existing designs and computer-aided engineering (CAE) tools such as CAD, FEA and SPICE models.

Then the control algorithms are tested against the requirements by simulating the model. The code is automatically generated from the simulation model for real-time testing of the control algorithms, and for real-time system simulation of hardware for testing the real microcontroller, field-programmable gate array or programmable logic controller. Finally, the code is automatically generated from the simulation model for implementing directly on the production targets.

Why use this approach?

As is evident from the above process, developers using a model-based